

UNIVERSIDAD CATÓLICA SANTO TORIBIO DE MOGROVEJO

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

DESARROLLO DE APLICACIONES MÓVILES

BANCO DE EJERCICIOS DE PRÁCTICA

Sesión 11 - Operaciones CRUD y Consultas RESTful

CampusGo · Preparación para la evaluación individual

Asignatura	Desarrollo de Aplicaciones Móviles
Unidad	Unidad II: Servicios web con acceso a base de datos y mecanismos de seguridad
Sesión de referencia	Sesión 11 - CRUD y consultas
Caso	CampusGo

Propósito: Practicar modificaciones sobre la API REST de CampusGo después del laboratorio. Los ejercicios no incluyen solución porque buscan desarrollar capacidad de análisis, implementación y sustentación técnica.

1. Indicaciones generales

- Trabaje sobre el proyecto CampusGo desarrollado en la sesión 11.
- Mantenga la arquitectura Route → Service → Repository.
- No coloque sentencias SQL dentro de routes.
- Obtenga el usuario autenticado desde JWT cuando la operación lo requiera.
- Utilice correctamente path parameters y query parameters.
- Use consultas parametrizadas con %s; no concatene valores provenientes del usuario dentro del SQL.
- Pruebe cada ejercicio en Postman con al menos un caso correcto y un caso de error.
- Prepárese para explicar por qué cada cambio se implementó en una capa determinada.
- No memorice código: practique el patrón de solución y sea capaz de adaptarlo a una variante.

Regla de estudio: Route recibe y responde · Service decide · Repository consulta.

2. Ejercicios propuestos

Ejercicio 1. Filtrar viajes por rango de precio

Requerimiento: Modifique GET /api/viajes para aceptar los query params opcionales precio_min y precio_max. Si ambos son enviados, solo deben mostrarse los viajes cuyo precio esté dentro del rango.

Condiciones mínimas:

- Validar que los parámetros sean numéricos.
- Si se envían ambos valores, precio_min debe ser menor o igual que precio_max.
- El filtro debe ejecutarse en SQL desde ViajeRepository.
- Ante parámetros inválidos, responder 400 Bad Request.

Microvariación de práctica: Permita que el usuario envíe solamente precio_max. ¿Qué tendría que cambiar?

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 2. Mostrar únicamente viajes con cupos disponibles

Requerimiento: Incorpore el query param solo_disponibles=true en GET /api/viajes. Cuando sea enviado, solo se deben retornar viajes con estado DISPONIBLE y cupos > 0.

Condiciones mínimas:

- No filtre los resultados recorriendo una lista en Python.
- El filtro debe formar parte de la consulta SQL.
- Si el parámetro no se envía, el endpoint debe conservar su comportamiento normal.

Microvariación de práctica: Cambie el requerimiento para mostrar solo viajes con 2 o más cupos disponibles.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 3. Consultar viajes por fecha mínima

Requerimiento: Modifique GET /api/viajes para aceptar el query param desde=YYYY-MM-DD. Debe retornar solo viajes cuya fecha sea igual o posterior a la indicada.

Condiciones mínimas:

- Validar el formato de fecha antes de consultar.
- Una fecha con formato inválido debe responder 400.
- La condición fecha >= valor debe implementarse en Repository.

Microvariación de práctica: Agregue también hasta=YYYY-MM-DD para consultar un rango de fechas.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 4. Consultar mis viajes publicados

Requerimiento: Implemente GET /api/viajes/mios para retornar únicamente los viajes publicados por el conductor autenticado.

Condiciones mínimas:

- Obtener usuario_id mediante get_jwt_identity().
- No recibir conductor_id mediante body, path ni query param.
- Si el usuario no tiene perfil de conductor, responder 403 Forbidden.

Microvariación de práctica: Permita filtrar mis viajes mediante ?estado=DISPONIBLE.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 5. Actualizar solamente los cupos de un viaje

Requerimiento: Implemente PATCH /api/viajes/{id}/cupos. El body deberá recibir únicamente el nuevo número de cupos.

Condiciones mínimas:

- Ejemplo de body: { "cupos": 3 }.
- Solo el conductor propietario del viaje puede modificarlo.
- No permitir cupos menores que cero.
- No permitir cambios en viajes FINALIZADO o CANCELADO.
- Use 403, 404 o 409 según corresponda.

Microvariación de práctica: ¿Qué cambiaría si los cupos permitidos estuvieran limitados entre 1 y 6?

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 6. Cambiar el estado de un viaje

Requerimiento: Implemente PATCH /api/viajes/{id}/estado para modificar el estado de un viaje del conductor autenticado.

Condiciones mínimas:

- Recibir el nuevo estado en JSON.
- Validar que el estado pertenezca al conjunto permitido.
- Controlar transiciones incoherentes; por ejemplo, un viaje CANCELADO no debe volver a DISPONIBLE.
- Ante conflicto de negocio, responder 409.

Microvariación de práctica: Defina qué transición debería ocurrir automáticamente cuando cupos llegue a cero.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 7. Evitar viajes duplicados

Requerimiento: Antes de registrar un nuevo viaje, valide que el mismo conductor no tenga otro viaje con la misma fecha, hora, origen y destino.

Condiciones mínimas:

- La verificación debe realizarse antes del INSERT.
- La consulta de existencia corresponde al Repository.
- La decisión de rechazar la operación corresponde al Service.
- Si ya existe, responder 409 Conflict.

Microvariación de práctica: Cambie la regla: se considera duplicado cualquier viaje del mismo conductor dentro de los siguientes 30 minutos.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 8. Detalle enriquecido de un viaje

Requerimiento: Modifique GET /api/viajes/{id} para incluir, además de los datos del viaje, nombres, apellidos y calificación del conductor.

Condiciones mínimas:

- Resolver la consulta mediante JOIN en ViajeRepository.
- No realizar varias consultas desde la Route.

- Si el viaje no existe, responder 404.

Microvariación de práctica: Agregue al response el nombre completo del conductor como un único campo conductor.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 9. Cancelar un viaje con restricciones

Requerimiento: Mejore DELETE /api/viajes/{id} para controlar la propiedad del viaje y su estado antes de cancelarlo.

Condiciones mínimas:

- No permitir cancelar un viaje de otro conductor.
- No permitir cancelar un viaje FINALIZADO.
- No eliminar físicamente el registro; cambiar estado a CANCELADO.
- Diferenciar correctamente 403, 404 y 409.

Microvariación de práctica: ¿Qué respuesta debería darse si el viaje ya estaba CANCELADO antes de recibir el DELETE?

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

Ejercicio 10. Reporte de viajes por estado

Requerimiento: Implemente GET /api/viajes/resumen para devolver la cantidad de viajes agrupados por estado para el conductor autenticado.

Condiciones mínimas:

- La agrupación debe realizarse en SQL mediante GROUP BY.
- El conductor debe obtenerse a partir del JWT.
- No realizar el conteo recorriendo los viajes en Python.
- La respuesta debe ser fácilmente consumible desde la aplicación móvil.

Microvariación de práctica: Amplíe el reporte para agrupar por estado y mes.

Evidencia mínima: captura o demostración en Postman del caso correcto y de al menos un caso de error; además, explicación de qué se modificó en Route, Service y Repository.

3. Matriz personal de avance

N.º	Ejercicio	Implementé	Probé errores	Puedo explicarlo
1	Filtrar viajes por rango de precio	☐	☐	☐
2	Mostrar únicamente viajes con cupos disponibles	☐	☐	☐
3	Consultar viajes por fecha mínima	☐	☐	☐
4	Consultar mis viajes publicados	☐	☐	☐
5	Actualizar solamente los cupos de un viaje	☐	☐	☐
6	Cambiar el estado de un viaje	☐	☐	☐
7	Evitar viajes duplicados	☐	☐	☐
8	Detalle enriquecido de un viaje	☐	☐	☐
9	Cancelar un viaje con restricciones	☐	☐	☐
10	Reporte de viajes por estado	☐	☐	☐

4. Preguntas de autoevaluación

- ☐ ¿Puedo explicar cuándo usar path parameter y cuándo usar query parameter?
- ☐ ¿Sé identificar qué validaciones pertenecen a Route y cuáles son reglas de negocio del Service?
- ☐ ¿Sé escribir una consulta parametrizada con filtros opcionales?
- ☐ ¿Puedo comprobar que el recurso pertenece al usuario autenticado?
- ☐ ¿Sé diferenciar cuándo corresponde 400, 401, 403, 404 y 409?
- ☐ ¿Puedo implementar un UPDATE parcial sin copiar una solución previa?
- ☐ ¿Sé justificar por qué DELETE en CampusGo se implementa como cancelación lógica?
- ☐ ¿Puedo modificar una consulta para agregar JOIN o GROUP BY?
- ☐ ¿Puedo adaptar un ejercicio si el docente cambia un valor, condición o filtro?
- ☐ ¿Puedo ejecutar, demostrar y explicar mi solución en pocos minutos?

Preparación para la evaluación: El objetivo no es memorizar diez soluciones. Debe dominar los patrones de CRUD, consultas, filtros, autorización, validación y códigos HTTP para poder resolver una variante del mismo tipo.

Practicar · Comprender · Modificar · Sustentar